

Reinforced Approximate Exploratory Data Analysis

Shaddy Garg¹, Subrata Mitra^{1*}, Tong Yu¹,
Yash Gadhia^{2†}, Arjun Kashettiwar²

¹ Adobe Research

² Indian Institute of Technology, Bombay

Abstract

Exploratory data analytics (EDA) is a sequential decision making process where analysts choose subsequent queries that might lead to some interesting insights based on the previous queries and corresponding results. Data processing systems often execute the queries on samples to produce results with low latency. Different downsampling strategy preserves different statistics of the data and have different magnitude of latency reductions. The optimum choice of sampling strategy often depends on the particular context of the analysis flow and the hidden intent of the analyst. In this paper, we are the first to consider the impact of sampling in interactive data exploration settings as they introduce approximation errors. We propose a Deep Reinforcement Learning (DRL) based framework which can optimize the sample selection in order to keep the analysis and insight generation flow intact. Evaluations with 3 real datasets show that our technique can preserve the original insight generation flow while improving the interaction latency, compared to baseline methods.

1 Introduction

Exploratory data analysis (EDA) is an interactive and sequential process of data understanding and insight generation where a user (e.g. a data analyst) issues an analysis action (i.e. a query) against a tabular data, receives some answers, subsection of the tabular data or some visualization and then decides which queries to issue next in order to understand the hidden characteristics of the data and associated insights better (???). Characteristics and insights obtained using EDA are crucial for subsequent decision making in various domains (?). The *success* of an EDA session and the quality of insights obtained from it large depend on two things: (1) how interactive the system is (i.e. how quickly results are obtained) and (2) whether the outcome of previous sequence of queries are reliable and representative enough for an expert analyst to issue subsequent queries so as to uncover interesting characteristics.

To improve latency of interactions, downsampling the data has been proposed in various works targeting approximate query processing (AQP) (???) and visualizations (???). For large datasets that are often a target for EDA (??), running

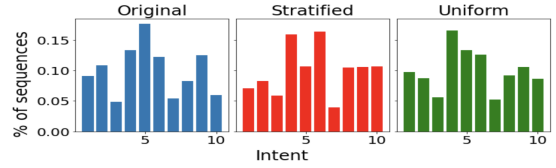


Figure 1: Divergence of intent distribution: Single sampling strategy (stratified or uniform) for all the contexts in sequential data exploration can make user to get mislead into wrong analysis flow due to approximation errors.

queries on samples can substantially reduce the query latency. However, sampling creates approximation errors and can mislead the users in an interactive data exploration flow, especially when the use, choice and degree of approximation is transparent to the user. In this paper we are the first to explore a setting where an EDA system *transparently* uses different forms of downsampling strategy on the original data to run the queries but protect the user from being mislead due to approximation error from poor choice of samples. Figure 1 shows distributions of 10 user-intents extracted from several thousand of realistic EDA sequences on Flights data. It can be observed that if only a particular downsampling strategy (e.g. stratified or uniform sampling) is used for every context in the entire sequence, the resulting intent distribution diverges from the ideal. This is because the results of the previous query can get distorted due to sampling and may prompt the user to a false path of analysis, which would have been useless if the original data was used. Second, there are numerous sampling techniques available, such as uniform sampling with different sampling rates, stratified sampling of few different kinds, systematic sampling, cluster sampling, diversity sampling etc. Each of these sampling algorithms are good at preserving some properties of the data at the cost of introducing approximation errors for other properties. For a sequential and interactive data exploration workflow where multiple different types of queries are used on different subsections of the data, there is often not a single sampling strategy that wins over everything else. This is because, in a sequential interactive exploration the amount of approximation in the results or visualizations generated from the previous queries, influences the user decision on what next query to run. Moreover, there is a trade-off between query latency and accuracy, when sampling is used.

*Corresponding Author

†Work done while at Adobe Research

Choosing a sample that gives higher approximation error (while making the query faster) can be desirable, as long as the approximation error does not mislead the user to make wrong decisions about subsequent analyses flow or alter the key takeaways.

This brings us to the second major problem that we handle in this paper. EDA being fundamentally an open-ended and expertise-driven process to understand a new dataset, there is no explicit *intent* that can be attributed to the analysis flow. Some recent papers looked at how to learn from EDA sequences performed by expert users in order to mimic those in a simulator to generate a sequence of queries and results on a new dataset, to help novice analysts (?). Similarly (?) provides suggestions to novice users about the next query to run based on learning from expert user sessions. While these works identify that exploratory analysis is not exactly random in nature and expert analysts usually follow few core implicit intents, but they do not handle how intent-aware learning can be performed in such situations. This lack of clear intent, as well as not having a clear measure of success or a implicit/explicit feedback at the end also makes our problem markedly distinct from other interactive settings such as task-oriented dialog systems (??), conversational recommender systems (??) and interactive retrieval systems (?). When developing the learning agent for these systems, it is usually assumed that the users have a clear intent, from which the loss function or reward function can be naturally derived. However, in our task the users usually have implicit intent, which makes development of the reward function non-trivial.

To jointly address the above two challenges, we propose an intent-aware deep reinforcement learning (DRL) framework, APPROXEDA, to enable the interactive exploratory data analysis in the presence of approximation errors. In this paper, we focus on how to prevent intent-divergence (i.e., approximation errors misleads users to a different intent). It is assumed that there is limited *intent-shift* (i.e., change of user’s core intent characteristics) in the problem setting. This assumption is widely adopted in other interactive applications (???), although it is interesting to address the intent-shift (?). As the first work on approximate exploratory data analysis in the interactive setting, we leave addressing *intent-shift* as future work.

Our contributions are summarized as follows.

- To our best knowledge, we are the first to identify an important yet practical problem where approximations should be used to improve interactivity in sequential exploration of data but can potentially mislead analysts to wrong analyses paths or intent-divergence.
- We make a case that depending on the context of the sequential analysis, different sampling strategies would be optimal for providing lower interaction latency while protecting intent-divergence.
- We novelly formulate the problem as a reinforcement learning task. Our formulation captures both the computational cost of executing the queries, as well as the sequential decisions made by users for different implicit intents. We model the choice of down-sampling strategy as the action-space and optimize the divergence of sequen-

tial interactions by an user along with the interactivity of the system, in the presence of approximations, in an intent-aware manner. To efficiently optimize the agent without explicit user intent, we develop a novel reward function and DRL learning stop criteria.

- We show the effectiveness of our technique by extensively evaluating our solution on 3 real-world datasets and against several baselines. We also present detailed ablation study and make our code and data available (?).

2 Background and Motivations

2.1 Motivating Example

In Figure 2(a.1), we show example of an EDA being performed on *all* the records of Flights data (?). At first an analyst attempts to understand the flights delay and filters the entries that had delay higher than average (with query Q1 and corresponding result/visualization R1). Then she wants to understand delays in detail by grouping those by *month* (i.e. Q2) and observes from the bar-plot (R2) that month of JUN contributed highest delays. Then she inspects by filtering the rows only for JUN (Q3 and R3). Not finding anything obvious she groups these filtered results by *origin* airport (Q4) and from the result (R4) concludes that delays in flights originating from LAX and ATL airports is the main culprit. Then she might forward this *insight* and investigate the root-cause of delays in these airports further. Please note: there is a lack of explicit intents (??) from the analysts corresponding to an EDA session.

Now in Figure 2(a.2), we illustrate the problem of arbitrary sample selection if sampled are used to speed-up queries but with approximate results. While nothing detrimental happens in the result R1 and the analyst would still run (Q2) to understand the delays grouped by *month*, a poor choice of sample may not highlight the large amount of delay contributed by the month of JUN. Therefore, not finding anything that stands out over different months, the analyst might go on to look for other anomalies or patterns of delay for different careers (Q3 and R3) and further not finding anything that suspicious, attempts to understand delays over different *days_of_week* (Q4 and R4) and concludes with a *misleading insight* that delays are primarily caused of flights on 4th day of the week. Here the analyst diverged from the ideal analysis flow (if original data was used) because of approximation errors in queries/results Q1/R1 and Q2/R2. She finally made a wrong decisions on her choice of subsequent queries at Q3. As a result overall EDA flow diverged.

Considering the above example, appropriate sub-sample selection is crucial based on the context of EDA at each step. Thus balancing low latency for good interactive experience vs. intent and insight preservation is the goal of this paper.

2.2 Sampling Strategies

A sample dataset T_s is a subset of rows from an original dataset T . Following different sampling strategies select these subsets differently with different set of parameters that control the size and statistics of T_s . We consider sampling strategies listed in Table 1 with different associated

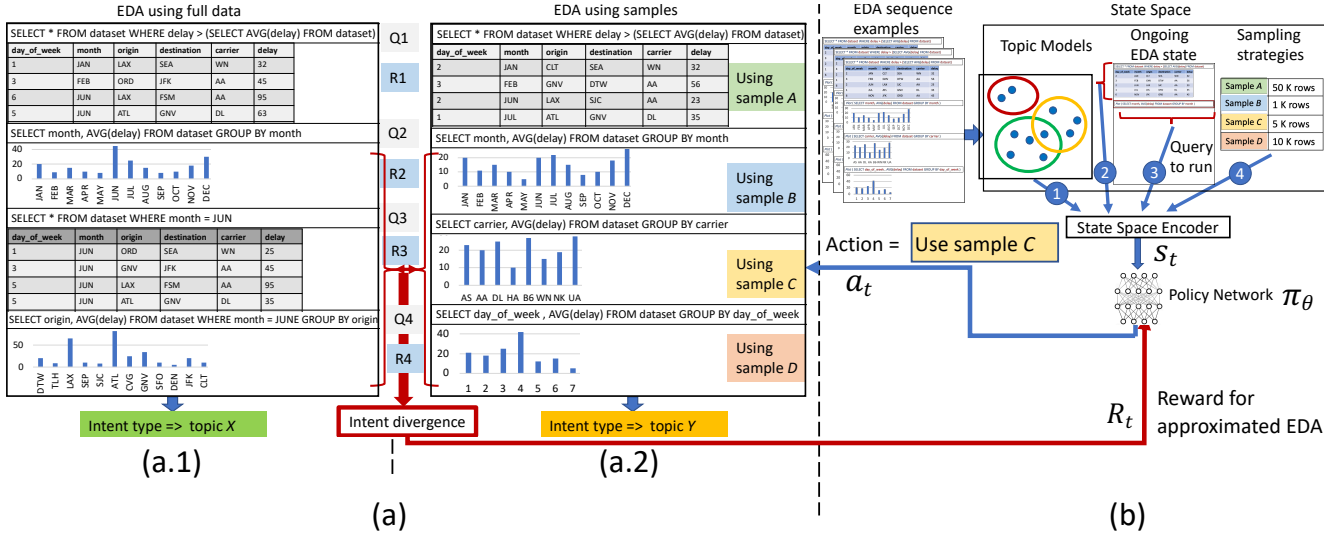


Figure 2: (a) Example of intent divergence between EDA sessions using full data (a.1) vs. using samples (a.2). Bad sample selection in (a.2) led to large errors in the result R2. Consequently, user diverted from ideal query sequence and finally found misleading insight in R4. (b) Workflow of our RL-based APPROXEDA is shown. The agent transparently interacts with a live EDA session (e.g. one on the left) and chooses the best action a_t , i.e. the optimal sample to use for the current so that intent divergence is minimized. APPROXEDA’s *State Space Encoder* combines 4 different information into a state s_t that is used by the policy-network (π_θ) to decide the best action.

Sample Name	Short Name	Parameters
Uniform	Uni@ τ	$\tau = [0.01, 0.05, 0.1]$
Systematic	Sys@ k	$k = [100, 20, 10]$
Proportional stratified	Strat@ τ	$\tau = [0.01, 0.05, 0.1]$
At most K stratified	Strat-K@ K	$K = [2k, 10k, 20k]$
Cluster	Clus@ k	$k = 10, \tau = [0.01, 0.05, 0.1]$
MaxMin Diversity	MaxMin@ k	$k = 0.1 * T $
MaxSum Diversity	MaxSum@ k	$k = 0.1 * T $

Table 1: Sampling strategies and corresponding parameters that together creates our action space. We use the *Short Name* to refer these.

parameters that control the sizes of the subsamples and consequently the amount of statistical information. More details of the sampling strategies is in Appendix ??.

3 Overview of Proposed Approach

In this paper we propose an intent-aware deep reinforcement learning (DRL)-based technique: APPROXEDA that can make the optimal choice of samples to be used at each step of the analysis flow to speed interactions but prevents intent-divergence.

3.1 Design of APPROXEDA

We show the high-level system architecture of APPROXEDA in Figure 2(b). APPROXEDA first takes in a history of EDA sequences performed by human data analysts and uses Topic Modeling technique to identify a set of implicate intents in those sequences. Then the RL-agent of APPROXEDA takes into account: (1) these clusters of these latent intents, (2) the history of the ongoing EDA exploration session including the query used so far and the corresponding display-outputs (graphs and dataframes), (3) the next query the user

is intending to run, and (4) the set of available samples created with different sampling strategies along with the size of each. The RL-agent, which is parameterized by a deep neural network (??), is trained *offline* to choose the optimal sampling strategy as the best *action* for different context of the analyses and intent. We considered alternate RL algorithms (DQN [(?)]) and REINFORCE [(?)]) but found the convergence and stabilization of A2C training (Lillicrap et. al), which is a hybrid between value and policy based approach, was better and reduces variance. The best action corresponding to each step, i.e. for each query in the EDA session attempts to minimize the divergence of intents due to approximation error caused by different samples, while optimizing for lower latency.

For our problem, lack of explicit intents in the EDA sessions, a moderately large action space of possible sampling strategies combined with the choice of parameters for each of those strategies that would control the statistical information preserved in the samples and associated trade-off for latency based on the size of the samples (# of rows), makes this a complex optimization problem. Appendix ?? further discusses why RL better choice instead of greedy solutions.

3.2 Intent Identification

The problem of intent aware sample selection requires us to first solve the problem of identifying the intent of the user given a query sequence Q_j . Similar to analyzing natural language texts to understand the topics, we can analyze query sequences to discover the user intents. Inspired by the previous work which uses topic modeling techniques to model goals that users carry out by executing a sequence of commands (?), we leverage topic modeling to discover the user intent, which is further used to derive the reward signal as

detailed later in Section 4.3.

Specifically, we use a bi-term model (BTM) (?) for the identification of the intent. Each topic output from the BTM model corresponds to an intent. For a query sequence Q_j , the output from the BTM model, ϕ is the probability distribution over all the topics.

$$\phi(Q_j) = \{P(t = t_i | Q = Q_j)\}_{i=1}^K \quad (1)$$

Here K is the number of topics, and each t_i corresponds to a topic. To get the corresponding intent, we take the *argmax* over the probability distribution. The obtained intent, I_{Q_j} is considered to be the user intent by APPROXEDA.

$$I_{Q_j} = \operatorname{argmax}(\{P(t = t_i | Q = Q_j)\}_{i=1}^K) \quad (2)$$

To decide the optimum number of topics for the BTM model, we use a coherence evaluation metric, the UCI measure (?), defined as

$$\text{score}_{UCI}(q_i, q_j) = \log \frac{p(q_i, q_j)}{p(q_i)p(q_j)}, \quad (3)$$

where $p(q_i)$ represents the probability of seeing a query q_i in a query sequence and $p(q_i, q_j)$ is the probability of observing both q_i and q_j co-occurring in a query sequence computed as follows, $p(q_i) = \frac{M(q_i)}{M}$ and $p(q_i, q_j) = \frac{M(q_i, q_j)}{M}$, where $M(q_i)$ is the count of query sequences containing the query q_i , $M(q_i, q_j)$ is the count of sequences containing both q_i, q_j and M is the total number of query sequences. The UCI score for an intent I , is calculated as, $\text{mean}\{\text{score}_{UCI}(q_i, q_j | q_i, q_j \in I, i \neq j)\}$ and the overall UCI score the average across all intents. Higher UCI score is indicative of better grouping because one would expect two queries belonging to the same intent to show up together regularly in the query sequences. We compute the overall UCI score for all $K \in [2, 15]$ and choose the best K .

4 RL Formulation

We use RL for intent-aware sample selection the agent learns the optimal policy modeled as a conditional distribution $\pi(a|s)$, specifying the probability of choosing action $a \in A$ when in state $s \in S$. If the agent chooses an action $a \in A$ at state $s \in S$, it receives a reward $r(s, a)$.

4.1 Action Space

A set of n samples are pre-created using different sampling strategies (and of different sizes). These samples are the set of n actions: $A = \{a_1, a_2, \dots, a_n\}_{i=1}^n$ for the RL-agent.

4.2 State Space

As shown in Figure 2 (b), we use 4 components while deciding which action to choose. These components form the state space of the RL agent. The agent at each timestep t for a query q_t has access to the ongoing EDA state comprising of the previous k queries $\{q_{t-1}, q_{t-2}, \dots, q_{t-k}\}$ and their result vectors $\{v_{t-1}, v_{t-2}, \dots, v_{t-k}\}$. Since reducing the latency is one of the primary goals of the agent, it also takes into account the latency encountered till timestep t . We denote this latency factor by $C_t = \sum_{i=0}^t c_i$, where c_i is the latency associated with running a query q_i on a sample a_i . Finally, to preserve the intent information of the sequence,

we also include the probability distribution over all the topics, $I_t = \phi(Q_t)$ as one of the state space components. Formally, the state space can be written as,

$$s_t = \{((q_t, v_t), (q_{t-1}, v_{t-1}), (q_{t-2}, v_{t-2})), I_t, C_t\} \quad (4)$$

Once the agent chooses an action $a_t \in A$, the agent moves to the next state s_{t+1} and returns a result (display vector) v_t , as the result of the query q_t . Our goal here is to choose a_t at each step, such that the overall latency for the sequence is minimised, simultaneously preserving the intent of the generated sequence. To do this we develop several reward functions and propose an intent-aware RL framework based on A2C methodology (?). Details of our training algorithm is given in supplement (??).

4.3 Reward Design

APPROXEDA calculates the reward at the end of each exploratory session. For the calculation of reward, we use a set of sequences generated on the original table and we refer them as ground truth, $O_i \in O$. Each O_i is a sequence of queries. Similarly, the sequence generated by the agent at the end of each episode is represented as Q_t . Reward design combines following components.

Latency Reward. Choosing samples of smaller size gives lower latency and hence this reward is calculated as:

$$R_{\text{latency}} = \sum_{t=1}^l r(s_t, a_t), \quad (5)$$

, where $r(s_t, a_t) = 1 - \frac{|T_s|}{|T|}$, where T_s is the time taken to run on a sample, T is the time taken to run on the original dataset, and l is the length of the sequence in the session. This reward helps to navigate the trade-off between lower approximation error vs. lower latency.

Intent Reward. Intent reward is designed to prevent divergence of EDA sequences due to approximations (§!??). To prevent intent-divergence we introduce two components in the intent reward. We denote the distance function for distance between two query sequences corresponding to two EDA sessions to be $\text{Distance}(Q_1, Q_2)$. We define the reward function to be,

$$R_{\text{dis}} = 1 - \text{Distance}(Q_t, Q_{\text{ground}}). \quad (6)$$

where Q_{ground} is the closest sequence to the given sequence Q_t in the original set O . We use EDA-Sim defined in (?) as distance metric. EDA-Sim compares the content and order of the results from the query sequence to calculate a similarity score. Based on this score we find a sequence from O that is closest with current sequence.

However, it is possible that two sequences may not have the exact same ordering or content (resulting in low EDA-Sim score) but still overall have same underlying intent. Hence we also use Euclidean Distance (*EuD*) to compare intent identified from topic model distributions (§ 3.2) as follows:

$$R_{\text{topic}} = 1 - \text{EuD}(\phi(Q_t), \phi(Q_{\text{ground}})) \quad (7)$$

Full intent reward is: $R_{\text{intent}} = R_{\text{dis}} + \delta * R_{\text{topic}}$.

Termination Reward. While the whole EDA session, comprising of query and corresponding visualization sequences, are important to the analyst, usually last few queries are

the most important as they provide key takeaway insights. Therefore, the another component of our reward is based on final insight preservation. The first subcomponent $R_{match} = \{0, 1\}$, a binary reward if the last k queries of the generated sequence Q_t match with at least one of the ground truth sequence belonging to the same intent cluster. However, often different query sequence lead to similar insights. Therefore, the other subcomponent R_{recall} as the top-k recall for the display vectors (results) from the generated sequence, Q_t and the closest sequence identified, Q_{ground} in Eqn. 6. Full termination reward is: $R_{term} = R_{match} + R_{recall}$.

Complete reward is $R_t = R_{latency} + \beta * R_{intent} + \gamma * R_{term}$. **Intent-divergence vs. intent-shift** This paper addresses the problem of *intent-divergence* where the users' have a set of consistent implicit objectives/intents during the EDA. While APPROXEDA deliberately introduces a bias towards a set of intents inferred from historical analysis sequences, this is justified as real expert users often follow some standard exploration strategies and look for similar types of insights even on new datasets as observed by multiple prior works (???) and do not perform random explorations. However, if such implicit objectives suddenly change causing an *intent-shift*, then APPROXEDA's current design would not be able to detect that. In this paper, it is assumed that there is limited *intent-shift* (i.e., change of user's core intent characteristics). This assumption is widely adopted in other interactive applications (???), although it is interesting to address the intent-shift (??). As the first work on approximate exploratory data analysis in the interactive setting, we leave addressing *intent-shift* as future work.

4.4 Implementation & Deployment

Algorithm 1: APPROXEDA's Training Algorithm

```

1: Train the ATENA §5.2 on given dataset  $D$ 
2: Generate sequences  $O$  using ATENA
3: Train a  $BTM$  model  $\phi$  with  $K$  topics on  $O$ 
4: Create sample set  $A = \{a_1, a_2, \dots, a_i\}_{i=1}^n$ 
5: Initialise A2C model
6: for  $i = 1, \dots, episodes$  do
7:   Initialize the state space  $s_t$  as in (4) with zeroes
8:   for  $t = 1, \dots, l$  do
9:     Generate a query  $q_t$  from the simulator
10:    Take action  $a_t$  for the given  $s_t$ 
11:    Update  $s_t$  with latest query  $q_t$ , display vectors  $v_t$ ,
        latency  $C_t$  and current intent as  $\phi(Q_t)$ 
12:   end for
13:   Calculate Individual rewards using §4.3.
14:   Calculate the total reward  $R_t$ 
15:   Calculate  $J_R(\pi) = \mathbb{E}_{a_t \sim \pi_\theta}[R_t]$ 
16:   Compute loss functions  $\mathbb{L}(\theta)$  for policy network
17:   Compute  $\mathbb{L}(\omega)$  for value network
18:   Update A2C model
19: end for

```

Algorithm 1 explains the steps we use for training our RL Agent. It takes in the dataset D , processes it by creating samples as shown in line 4 and training ATENA simulator on the same as shown in line 1. Then on the generated sequences, a BTM model is trained in line 3. Once this is done, we then

Sampling Strategy	No. of Rows	Effective SR
Uni@1%	5232	0.01
Uni@5%	26155	0.05
Uni@10%	52312	0.1
Sys@100	5231	0.01
Sys@20	26154	0.05
Sys@10	52311	0.1
Clus@1%	5230	0.01
Clus@5%	26154	0.05
Clus@10%	52306	0.1
MinMax@5%	27202	0.052
MaxSum@5%	25109	0.048
KStrat1@2k	14124	0.027
KStrat1@10k	28771	0.055
KStrat1@20k	57542	0.11
KStrat2@2k	9416	0.018
KStrat2@10k	25125	0.048
KStrat2@20k	47080	0.09
Strat1@1%	5220	0.01
Strat1@5%	26146	0.05
Strat1@10%	52375	0.1
Strat2@1%	5232	0.01
Strat2@5%	26148	0.05
Strat2@10%	52320	0.1
Strat3@1%	5240	0.01
Strat3@5%	26150	0.05
Strat3@10%	52315	0.1
Strat4@1%	5235	0.01
Strat4@5%	26149	0.05
Strat4@10%	52321	0.1

Table 2: Details of Samples generated on Flights Dataset

Sampling Strategy	No. of Rows	Effective SR
Uni@1%	3242	0.01
Uni@5%	16210	0.05
Uni@10%	32419	0.1
Sys@100	3241	0.01
Sys@20	16209	0.05
Sys@10	32419	0.1
Strat1@1%	3246	0.01
Strat1@5%	16225	0.05
Strat1@10%	32425	0.1
Strat2@1%	3238	0.01
Strat2@5%	16215	0.05
Strat2@10%	32426	0.1
Strat3@1%	3240	0.01
Strat3@5%	16208	0.05
Strat3@10%	32420	0.1
Strat4@1%	3248	0.01
Strat4@5%	16206	0.05
Strat4@10%	32425	0.1

Table 3: Details of Samples generated on Housing Dataset

Sampling Strategy	No. of Rows	Effective SR
Uni@1%	4266	0.01
Uni@5%	21334	0.05
Uni@10%	42670	0.1
Sys@100	4266	0.01
Sys@20	21334	0.05
Sys@10	42669	0.1
Strat1@1%	4267	0.01
Strat1@5%	21336	0.05
Strat1@10%	42672	0.1
Strat2@1%	4235	0.01
Strat2@5%	21330	0.05
Strat2@10%	42664	0.1
Strat3@1%	4235	0.01
Strat3@5%	21340	0.05
Strat3@10%	42667	0.1
Strat4@1%	4237	0.01
Strat4@5%	21367	0.05
Strat4@10%	42673	0.1

Table 4: Details of Samples generated on Income Dataset

start the training of our agent as demonstrated in lines 5-18 of Algorithm 1. We follow episodic training where we calculate rewards at the end of each episode as shown in line 13. Once we calculate the total reward, we compute the loss functions and update both actor (policy) and critic (value) networks as explained in §???. Appendix ?? discusses deployment aspects.

5 Experiments and Analysis

Our experiments aim to answer these:

- **RQ1.** Can APPROXEDA prevent intent-divergence by context-aware sample selection? (Results in § 5.3)
- **RQ2.** Can approximate EDA with APPROXEDA still captures the final insights as the original? (Results in § 5.4)
- **RQ3.** Can APPROXEDA provide significant latency reduction while addressing RQ1 and RQ2? (Results in § 5.5)

5.1 Experimental Setup

All experiments used a 32 core Intel(R)Xeon(R) CPU E5-2686 with 4 Tesla V100-SXM2 GPU(s). Model and implementation details are in supplement ??.

Datasets. Table 5 summarizes the popular and public datasets: Flight (?) (also used by (?)), Housing (?) and Income (?)

Actions In total we use 29 actions based on 6 sampling

strategies and corresponding parameter combinations as presented in Table 1. Effective SR denotes the sampling rate defined as total # rows in the sample / # rows in the full data. Table 2, 3, 4 show the effective sampling rate and the number of rows for different samples for each dataset.

Baselines.

- **BLINKDB** : BLINKDB (?) attempts to minimize error for each query by selecting the stratified-sample that has the best overlap for QCS (explained before) of the incoming query to be executed. When no overlap with stratified samples, we use 1% uniform sample.
- **CIGREEDY** : We develop another intelligent baseline that first calculates a confidence interval (CI) for aggregate value requested by the incoming query on all the available samples (using a closed form formulation from (?)). Then chooses sample with the tightest CI at 95% (i.e. indicating more confidence in the result).
- **Uni@10%** and **Uni@1%**: We compare our results with two other baselines where the agent always chooses a Uniform 10% sample and Uniform 1% sample respectively.

Parameters. We use $K = 4$ intents from BTM as it maximizes overall UCI score. Appendix ?? and Figure ?? shows the clusters and UCI scores. We use values of β and γ as 1 to provide equal weightage to rewards after scaling them.

5.2 Evaluation Methodology

EDA Interactions using ATENA. There are no public datasets available that combines both exploratory insight generation query sequences and the corresponding results on a given data that is also publicly available. Moreover, similar to many other interactive scenarios, such as conversational recommender systems (?), dialog systems (??) and dialog-based interactive image retrieval (??), a major challenge in the evaluation is that we need to have access to user reactions to any possible results returned by the system, which are exhaustive to collect in practice. Therefore, similar to (????), we adopt a setting where the user reactions are generated by simulators as surrogates for real users. We overcome this by using an EDA simulator from prior work (?), called ATENA that is trained using real analysis sequences from expert analysts, which uses high-level statistical characteristics of the data along with its schema structure to generate realistic analysis sequence patterns similar human experts with various implicit intents (?). For each of the datasets, we train this simulator following the methodology described in (?). Then we use this trained stochastic simulator to generate data exploration sessions which constitute sequence of queries and corresponding results or visualizations. For each dataset several such unique sequences can be generated by the simulator using multiple runs and is summarized in Table 5. For each dataset, we set aside $1k$ sequences generated by the simulator using full data as held-out set for evaluations. For APPROXEDA and the baselines, we also generate $1k$ sequences by letting these methods interact with the simulator where for each query of the EDA, these methods select the optimal samples using their respective algorithms. Each of the sequences generated from any of the methods are fed into the BTM model to get the probability distribution of intents over the topics.

5.3 Evaluation: Intent Divergence (RQ1)

We evaluate APPROXEDA compared to the baselines in preserving the intent distributions (distribution of EDA interaction sequences over topic-models) of the original EDA sequences created without using any samples for approximations. In Figure 3 we present Euclidean Distance (ED) (normalized w.r.t. APPROXEDA) between the intent distributions resulting from one of the baselines and from original unapproximated scenario. Lower ED is desirable as it indicates better matching of intent distribution. We can see that for Flights and Income, APPROXEDA is best in preserving the distribution. For Housing it Uni@10% (uniform sample at 10%) and CIGREEDY does better. This is because both Uni@10% (uniform sample at 10%) and CIGREEDY ends up choosing large overall sample sizes, and hence less information loss and approximation error. However, in the following § 5.5 and associated latency improvement plot in Figure 5, we see that such use of large samples makes these methods extremely poor for latency reduction and defeats the purpose of using samples to speed up interactivity. Supplement (A.1) provides the exact number of rows corresponding to each samples (actions) created by various sampling strategies.

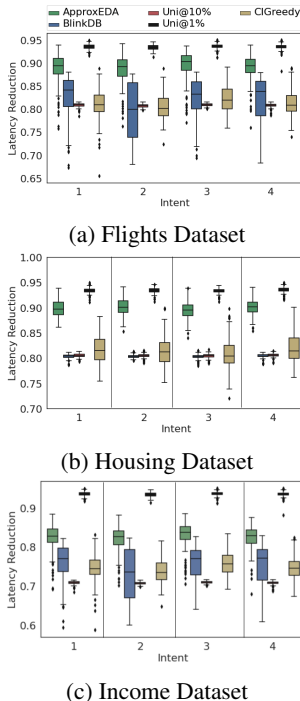
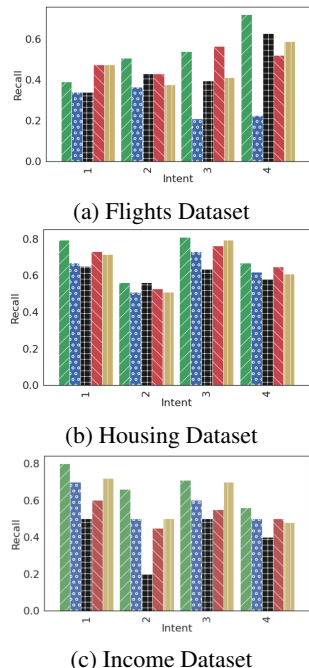
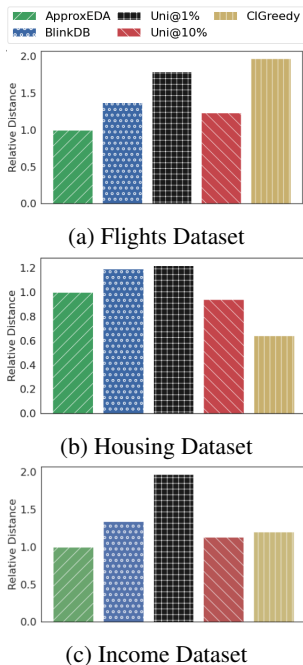
5.4 Evaluation: Insight Preservation (RQ2)

We evaluate how well the final insight of the EDA is preserved in addition to preserving the intent distribution. Analysts usually derive insights at the end of each analysis sequence, using the outcome of the final few queries. Since we do not have explicit labels about whether user actually received satisfactory insights at the end, we calculate a recall value based on results of row obtained from different sampling based methods w.r.t. results obtained from use of full data. To calculate this recall, we use top $k = 5$ result rows obtained from last 2 queries from each of the analysis sequence – a thumb rule we understood by talking to experts.

$$Recall_{Intent=I} = \frac{|\bigcup_{i=1}^n M(Q_i) \cap \bigcup_{j=1}^m M(O_j)|}{|\bigcup_{j=1}^m M(O_j)|}, Q_i, O_j \in I \quad (8)$$

where Q_i refers to the generated sequence, O_j refers to the sequence generated on full data and $M(Q_i)$ is the union of the top k results of the last two queries of the sequence Q_i .

Please note: the intent distribution (§ 5.3) along with this final insight preservation, as a combination ensures that users' do not diverge from their analysis workflow. This is because there can be hypothetical solutions that might directly match the results from last few queries, without replicating the intent of the analysis captured through the entire sequence of analysis. Even though, such a solution would provide a high recall, but will fail to preserve the purpose of data exploration and associated understanding consumed by analysts (?). From Figure 4 we see that for 2 out of 4 intents for Flights data and all of the intents for Housing data and Income data, APPROXEDA provides highest recall compared to the baselines. As explained in § 5.3, the reason either Uni@10% or CIGREEDY sometimes provide higher recall because they end up choosing large sample sizes, which in turn lead to much higher latency (Figure 5). We also compute



Dataset	# rows in full data	Unique sequences
Flights	5231403	10695
Housing	324196	6016
Income	426753	4032

Model	Relative ED	Latency Reduction	Overall Mean Recall			
			Intent 1	Intent 2	Intent 3	Intent 4
APPROXEDA	1	0.89	0.32	0.74	0.29	0.63
APPROXEDA - R_{term}	1.08	0.92	0.39	0.60	0.32	0.38
APPROXEDA - R_{intent}	1.36	0.91	0.31	0.57	0.30	0.35
APPROXEDA - $R_{latency}$	1.12	0.83	0.28	0.68	0.29	0.68

Action Space	Relative ED	Latency Reduction	Overall Mean Recall			
			Intent 1	Intent 2	Intent 3	Intent 4
Only Uniform	1.34	0.90	0.41	0.60	0.28	0.48
Uniform + Stratified	0.85	0.84	0.34	0.58	0.30	0.65
Uniform + Stratified + Cluster	1.12	0.90	0.41	0.60	0.30	0.66
All Samples	1	0.89	0.32	0.74	0.29	0.63

an *overall recall* corresponding to all the results returned by the last 2 queries and illustrate in ??.

5.5 Evaluation: Latency Reduction (RQ3)

In Figure 5 we compare the latency improvement by APPROXEDA compared to the other baselines. We compute the overall latency at a sequence level by summing up latency for each individual queries in that sequence. We calculate % of reduction in sequence-level latency compared to the latency when each query execute against full data (y-axis: higher is better). We show box-plots to show the distribution of values for each sequences, grouped by different intents. It can be observed that median sequence-level latency reduction for APPROXEDA is almost 90% (that is the analysis sequences take only $1/10^{th}$ of the original time. Note: even though Uni@1% provides most reduction as it always uses 1% sample of the data irrespective of the context. Recall from Figure 3 and 4, Uni@1% is pretty bad in terms of pre-

serving the intent distribution or the final insights. The combination of RQ1, RQ2 and RQ3 shows that APPROXEDA’s state can capture different context of sequential analysis and choose optimum trade-off space between statistical effectiveness of different samples vs. corresponding latencies.

5.6 Qualitative Evaluation

Due to space, in Appendix ?? Figure ?? we show a real EDA session with APPROXEDA to illustrate in-spite of it selecting frugal samples at various points (e.g. Clus@1%, Uni@5% etc.), APPROXEDA can mostly preserve the query structure, analysis sequence and final outcome/insight. In Figure 6 we show the variation of APPROXEDA’s choice of different actions (selection of samples) for 4 different intents identified for the Flights data across all the held out sequences. We found that certain samples (e.g. MaxMin@5%, Sys@20%, Strat1 @1%) were predominantly chosen for certain intents (e.g. #4). Intuition is that such samples can pre-

serve representations of smaller groups better, which is necessary for that intent preservation.

5.7 Ablation Study

We conduct an ablation study to illustrate the effect of each component on the final metrics in Table 6. The show Relative-ED, Latency-Reduction and Mean-Overall-Recall as the metric. We progressively remove one reward at a time and see the effect it has on the metrics. Comparing APPROX-EDA and APPROXEDA - R_{term} , we observe that the latency reduction increases. This is because $R_{latency}$ now has a greater impact on the total reward. However, this is accompanied by a sharp decrease in the Mean Recall value for Intent 2 and Intent 4 as this metric is highly dependant on R_{term} . The increase in mean recall for Intent 1 and Intent 3 is nominal. Removing the R_{intent} , we notice that the model now performs worse on every metric except latency. This is because both Relative ED and Mean Recall are highly intent dependant and the lack of intent reward makes it difficult for model to learn a good enough distribution. The improvement in latency can be credited to the higher weight of the latency reward as seen in earlier the earlier case. We then verify the effect of removing $R_{latency}$. As expected, we observe that the latency reduction has largely dropped and thus the agent trained without $R_{latency}$ is free to choose larger samples in order to maximise the reward. We notice that, the model now performs similar to APPROXEDA and is better in terms of Metric Recall on Intent 4. Table 7 shows sensitivity of APPROXEDA w.r.t. to availability of different groups of sampling algorithms in its action space.

Model Parameter	Relative ED	Latency Reduction	Mean Overall Recall			
			Intent 1	Intent 2	Intent 3	Intent 4
$\delta = 1$	1	0.89	0.32	0.74	0.29	0.68
$\delta = 0$	1.2	0.90	0.31	0.61	0.3	0.42

Table 8: Ablation study: Variation of δ in $R_{intent} = R_{dis} + \delta * R_{topic}$ (Ref. § 4.3).

We also do an ablation study on δ and show the same in Table 8. We observe that both components of our reward are important. Setting $\delta = 0$ increases our relative distance between distributions. Although it reduces the latency, the gain is small. We also see that for various intents, our recall decreases significantly for Intent 2 and Intent 4. This is because setting $\delta = 0$ removes the component that matches intent of the original sequence with the generated.

6 Related Works

Intent Analysis and Understanding: Several papers dealt with the concept of intents or goals for recommendations in process mining, web mining, education (??) and analyze patterns in user behavior from application log/clicks data (??) or characterised SQL query logs to understand analysis behaviors (?). Distinct from all these, we identify goal as a combination of queries and the corresponding results in a sequential decision making workflow.

Data Exploration: (?) simplified the manual creation EDA notebooks, citechirigati2016data focused on related dataset recommendation. (?) attempts to flag false discoveries in EDA automation but does not consider approximations or

intent of analysts. Recent work also focused on automatically generating data insights (??) or creating or augmenting EDA workflows (???). Several papers used samples and approximate query processing to speed up data explorations and analytics (?????) but we are the first to look at the sequence and context of such exploration for visual analytics.

Interactive Applications by Reinforcement Learning ?? propose interactive dialog systems based on reinforcement learning. ? propose a dialog-based interactive image retrieval system involving natural language feedback on visual attributes of the items. ? propose reinforcement learning with a drill-down method to interactively retrieve complex scene images. Reinforcement learning from reformulations is proposed for question answering, by modeling the answering process as multiple agents walking in parallel on the knowledge graph (?). ? propose a meta reinforcement learning method in complex question answering, which quickly adapts to unseen questions. Reinforcement learning has also been applied to elicit user preferences in conversational recommender systems (???). Similar to these interactive scenarios (???), a major challenge in the evaluation is that we need to have access to user reactions to any possible results returned by the system, which are exhaustive to collect. Therefore, similarly, we adopt a setting where the user reactions are generated by simulators as surrogates for real users as detailed in Section 5.2.

7 Conclusions

We proposed a deep RL based technique to optimize exploratory data analytics using down-sampled data. Our intelligent and contextual sample selection can prevent misleading explorations and insights while still ensuring lower latency of interactions. Evaluations show that our method is superior to baselines when used on three real-world dataset. Expedita doloribus commodi aut quod distinctio provident, neque perferendis in sed repellat asperiores placeat illo adipisci excepturi, totam suscipit sed expedita iusto nulla numquam optio dolores odit, officia consequuntur reprehenderit facilis fugit illum amet placeat veritatis nemo, qui suscipit officia rem quidem?Quos enim eos deserunt repudiandae hic consequatur inventore aut quaeat doloreque, praesentium natus deleniti minima totam fuga distinctio maxime nesciunt, consectetur assumenda quos temporibus consequatur voluptatem quidem optio nulla ratione, officia pariatur dolorum qui veniam, assumenda delectus facilis iure quisquam aperiam minima?Et fuga assumenda hic eligendi similique optio nemo quibusdam eum voluptatibus consequuntur, ullam nisi laboriosam natus eaque consequuntur voluptatibus asperiores quod magni?Modi laborum excepturi veritatis velit quos similique ut, dicta quisquam numquam culpa, aspernatur adipisci corrupti nihil quo impedit enim suscipit earum nesciunt quis aut, alias perspicatis assumenda ut neque et minus consequuntur repellendus at labore quam, deserunt eaque nihil iste.Praesentium nam delectus quasi ad enim esse non doloribus voluptates, veritatis vel quo aliquam pariatur?Quia molestiae non aliquam, vel laboriosam culpa architecto autem distinctio atque commodi officia reiciendis inventore similique, nisi non inventore rem quis quibusdam voluptas, eum repudiandae officiis

consequuntur unde deserunt laborum tempora consecetur
voluptates modi natus?Repellendus ea vero rem temporibus
quia officia, ipsa saepe repellendus commodi quos debitis